

```

1 import tensorflow as tf
2 from tensorflow.keras.utils import to_categorical
3 from tensorflow.keras.datasets import cifar10
4 from tensorflow.keras.preprocessing.image import ImageDataGenerator
5 from tensorflow.keras.applications import ResNet50
6 from tensorflow.keras.layers import Dense, Flatten, GlobalAveragePooling2D, Dropout, BatchNormalization, UpSampling
7 from tensorflow.keras.models import Model
8 from tensorflow.keras.callbacks import EarlyStopping, ReduceLROnPlateau
9 (x_train, y_train), (x_test, y_test) = cifar10.load_data()
10 from sklearn.model_selection import train_test_split
11 x_train = x_train.astype('float32')
12 x_test = x_test.astype('float32')
13 x_train = tf.keras.applications.resnet50.preprocess_input(x_train)
14 x_test = tf.keras.applications.resnet50.preprocess_input(x_test)
15 from sklearn.model_selection import train_test_split
16 x_train, x_val, y_train, y_val = train_test_split(x_train, y_train, test_size=0.1)
17 y_train = to_categorical(y_train)
18 y_test = to_categorical(y_test)
19 y_val = to_categorical(y_val)
20 train_generator = ImageDataGenerator(
21     rotation_range=10,
22     width_shift_range=0.1,
23     height_shift_range=0.1,
24     horizontal_flip=True,
25     shear_range=0.1,
26     zoom_range=0.1
27 )
28 val_generator = ImageDataGenerator(
29     rotation_range=10,
30     width_shift_range=0.1,
31     height_shift_range=0.1,
32     horizontal_flip=True,
33     shear_range=0.1,
34     zoom_range=0.1
35 )
36 test_generator = ImageDataGenerator(
37     rotation_range=10,
38     width_shift_range=0.1,
39     height_shift_range=0.1,
40     horizontal_flip=True,
41     shear_range=0.1,
42     zoom_range=0.1
43 )
44

```

```

1 from IPython.core import history
2 from os import name
3 inputs = Input(shape=(32, 32, 3))
4 resized_inputs = UpSampling2D(size=(7, 7))(inputs)
5 resnet_model = ResNet50(
6     include_top=False,
7     weights='imagenet',
8     input_tensor=resized_inputs
9 )
10 for layer in resnet_model.layers:
11     layer.trainable = False
12 NUM_LAYERS_TO_UNFREEZE = 40
13 for layer in resnet_model.layers[-NUM_LAYERS_TO_UNFREEZE:]:
14     layer.trainable = True
15 x = GlobalAveragePooling2D()(resnet_model.output)
16 x = Dense(1024, activation='relu')(x)
17 x = BatchNormalization()(x)
18 x = Dropout(0.3)(x)
19 x = Dense(512, activation='relu')(x)
20 x = BatchNormalization()(x)
21 x = Dropout(0.2)(x)
22 x = Dense(256, activation='relu')(x)
23 x = BatchNormalization()(x)
24 x = Dropout(0.2)(x)
25 x = Dense(128, activation='relu')(x)
26 x = BatchNormalization()(x)
27 x = Dropout(0.2)(x)
28 classification_output = Dense(10, activation='softmax', name='classification')(x)
29 ResNet_model = Model(inputs=inputs, outputs=classification_output, name="Resnet")
30 optimizer = tf.keras.optimizers.Adam()

```

```
31 ResNet_model.compile(  
32     optimizer=optimizer,  
33     loss='categorical_crossentropy',  
34     metrics=['accuracy']  
35 )  
36 num_layers=len(ResNet_model.layers)  
37 print(num_layers)  
38 reduce_lr=ReduceLROnPlateau(monitor='val_loss',factor=0.5,patience=1,min_lr=0.001)
```

188

```
1  batch_size=256  
2  epochs=5  
3  history=ResNet_model.fit(  
4      train_generator.flow(x_train,y_train,batch_size=batch_size),epochs=epochs,steps_per_epoch=x_train.shape[0],  
5      [reduce_lr,early_stopping],shuffle=True)  
6  test_loss=test_accuracy=ResNet_model.evaluate(test_generator.flow(x_test,y_test,batch_size=batch_size))  
7  print("test loss:",test_loss[0])  
8  print("test accuracy:",test_accuracy[1])
```

Epoch 1/5

40/175  1:51 825ms/step - accuracy: 0.5533 - loss: 1.4455